



Sistema de Gestão Comercial para Pequenos Negócios (SGC) -
Documentação:

THE Way SUPPLEMENTS

Alunos: João Francisco Torres e Marcus Vinícius Rodrigues Bacelar;
Disciplina: Desenvolvimento Web;
Professor: Felipe Pires Ferreira.

Sumário

1. Descrição do Sistema	2
2. Requisitos Funcionais	3
2.1 Autenticação.....	3
2.2 Gestão de Clientes.....	3
2.3 Gestão de Produtos.....	3
2.4 Registro de Vendas.....	3
2.5 Relatórios.....	4
2.6 Recomendação de Suplementos.....	4
3. Requisitos não funcionais	5
3.1 Segurança.....	5
3.2 Arquitetura.....	5
3.3 Comunicação da Aplicação.....	5
3.4 Interface do Sistema.....	5
3.5 Manutenibilidade.....	5
3.6 Controle de Versão.....	5
4. Arquitetura proposta	6
5. Padrões de projeto escolhidos	7
<u>Referências e Links</u>	8

1. Descrição do sistema

O mercado de suplementos alimentares tem crescido significativamente nos últimos anos, aumentando também a demanda por sistemas que auxiliem na gestão eficiente desse tipo de negócio. O sistema “**TheWay**(SGC)” é uma aplicação web desenvolvida para auxiliar na gestão comercial de uma loja de suplementos alimentares.

Muitas lojas desse segmento enfrentam dificuldades no controle eficiente de clientes, produtos, estoque e vendas, especialmente quando essas informações são gerenciadas manualmente ou em sistemas pouco integrados. Além disso, muitos consumidores possuem dúvidas sobre quais suplementos são mais adequados para seus objetivos, como ganho de massa muscular, emagrecimento ou melhora de desempenho físico.

Diante disso, o sistema permitirá o cadastro de clientes, gerenciamento de produtos, controle de estoque e registro de vendas, centralizando todas as informações em uma única plataforma e facilitando a administração da loja. Como funcionalidade adicional, o sistema também contará com um módulo de recomendação de suplementos, que poderá ser utilizado tanto por clientes quanto por funcionários da loja. A partir do objetivo informado pelo cliente, como hipertrofia, emagrecimento ou melhora de performance, o sistema poderá apresentar uma lista de suplementos recomendados, auxiliando no processo de escolha dos produtos.

Além disso, a aplicação oferecerá relatórios de vendas por período e por cliente, permitindo que os administradores tenham uma visão estratégica do desempenho do negócio. A solução será construída utilizando arquitetura em camadas, API REST, autenticação via JWT e interface web desenvolvida em Django, garantindo organização do código, segurança e facilidade de manutenção.

2. Requisitos Funcionais

2.1 Autenticação

- **RF01** – O sistema deve permitir que usuários realizem login utilizando username e senha.
- **RF02** – O sistema deve gerar um token de autenticação JWT após o login.
- **RF03** – O sistema deve permitir controle de acesso por perfil de usuário (ADMIN ou FUNCIONARIO).

2.2 Gestão de Clientes

- **RF04** – O sistema deve permitir cadastrar clientes com nome, CPF, email, telefone e endereço.
- **RF05** – O sistema deve permitir consultar clientes cadastrados.
- **RF06** – O sistema deve permitir atualizar dados de clientes.
- **RF07** – O sistema deve permitir remover clientes, desde que não possuam vendas registradas.

2.3 Gestão de Produtos

- **RF08** – O sistema deve permitir cadastrar produtos com nome, descrição, preço e quantidade em estoque.
- **RF09** – O sistema deve permitir consultar produtos cadastrados.
- **RF10** – O sistema deve permitir atualizar informações de produtos.
- **RF11** – O sistema deve permitir remover produtos do sistema.

2.4 Registro de Vendas

- **RF12** – O sistema deve permitir registrar vendas associadas a um cliente e a um usuário responsável.
- **RF13** – O sistema deve permitir adicionar itens à venda contendo produto, quantidade e preço unitário.
- **RF14** – O sistema deve calcular automaticamente o valor total da venda.
- **RF15** – O sistema deve atualizar automaticamente o estoque após a realização da venda.
- **RF16** – O sistema não deve permitir registrar vendas sem itens.

2.5 Relatórios

- **RF17** – O sistema deve permitir visualizar vendas registradas.
- **RF18** – O sistema deve permitir gerar relatórios de vendas por período.
- **RF19** – O sistema deve permitir visualizar o histórico de compras de um cliente.

2.6 Recomendação de Suplementos

- **RF20** – O sistema deve permitir cadastrar objetivos de suplementação (ex: hipertrofia, emagrecimento, desempenho).
- **RF21** – O sistema deve permitir associar produtos a objetivos específicos.
- **RF22** – O sistema deve permitir sugerir suplementos com base no objetivo informado pelo cliente.

3. Requisitos não Funcionais

3.1 Segurança

- **RNF01** – O sistema deve utilizar autenticação baseada em Token JWT para garantir a segurança no acesso aos endpoints da API.
- **RNF02** – As senhas dos usuários devem ser armazenadas utilizando criptografia por hash, garantindo a proteção das credenciais

3.2 Arquitetura

- **RNF03** – O sistema deve ser desenvolvido seguindo o padrão de arquitetura em camadas, separando responsabilidades entre apresentação, lógica de negócio e acesso a dados.

3.3 Comunicação da Aplicação

- **RNF04** – A aplicação deve disponibilizar uma API REST, permitindo comunicação entre os componentes do sistema utilizando o formato JSON.

3.4 Interface do Sistema

- **RNF05** – A interface do sistema deve ser acessível por meio de navegadores web, garantindo facilidade de acesso para os usuários.
- **RNF06** – A interface web do sistema deve ser desenvolvida utilizando o framework Django.

3.5 Manutenibilidade

- **RNF07** – O código do sistema deve ser organizado de forma modular, facilitando manutenção, evolução e escalabilidade da aplicação.

3.6 Controle de Versão

- **RNF08** – O sistema deve utilizar controle de versão por meio do GitHub, permitindo rastreamento de alterações e colaboração entre os desenvolvedores.

4. Arquitetura Proposta

O sistema “**TheWay**(SGC)” será desenvolvido utilizando o modelo de arquitetura em camadas, que organiza a aplicação em diferentes níveis de responsabilidade, facilitando manutenção, escalabilidade e organização do código.

A aplicação será dividida principalmente nas seguintes camadas:

Camada de Apresentação -

Responsável pela interface gráfica do sistema, que permitirá que os usuários interajam com a aplicação por meio de navegadores web. Essa camada será desenvolvida utilizando o framework Django, que consumirá os serviços disponibilizados pela API para exibir informações e permitir a realização de operações como cadastro de clientes, gerenciamento de produtos e registro de vendas.

Camada de Aplicação(API)-

Responsável por disponibilizar os endpoints da API REST, que serão utilizados pela interface web para acessar as funcionalidades do sistema. Essa camada será responsável por receber as requisições dos usuários, validar os dados recebidos e encaminhar as operações para a camada de lógica de negócio.

Camada de Lógica de Negócio -

Contém as regras de negócio do sistema, como controle de estoque, cálculo automático do valor das vendas, validação de regras relacionadas a clientes e produtos, além da lógica responsável pelo módulo de recomendação de suplementos com base nos objetivos informados pelos clientes.

Camada de Persistência de Dados -

Responsável pela comunicação com o banco de dados, realizando operações de armazenamento, consulta, atualização e remoção de dados. Essa camada manipulará as entidades do sistema como usuários, clientes, produtos, vendas e itens de venda.

Além disso, o sistema utilizará autenticação baseada em JSON Web Token (JWT), para garantir a segurança no acesso às rotas da API, permitindo controle de acesso baseado em perfis de usuário (ADMIN e FUNCIONARIO).

Essa organização em camadas contribui para a separação de responsabilidades, tornando o sistema mais organizado, seguro e fácil de manter ao longo do tempo.

5. Padrões de Projetos Utilizados

Para garantir uma melhor organização do código e facilitar a manutenção do sistema, serão utilizados alguns padrões de projeto durante o desenvolvimento do sistema “**TheWay**(SGC)”.

Arquitetura em Camadas -

O sistema seguirá o padrão de arquitetura em camadas, separando a aplicação em diferentes níveis de responsabilidade, como apresentação, lógica de negócio e acesso a dados. Esse padrão facilita a manutenção do código, melhora a organização do sistema e permite que alterações em uma camada tenham menor impacto nas demais.

Padrão MVC (Model–View–Controller) -

O framework Django utiliza o padrão arquitetural MVC, no qual o sistema é dividido em três componentes principais:

- Modelo: representa as entidades do sistema e a estrutura dos dados armazenados no banco de dados.
- Visão: responsável pela apresentação das informações ao usuário na interface web.
- Controlador: responsável por receber as requisições, processar os dados e coordenar as ações entre modelo e visão.

Esse padrão permite uma melhor separação entre interface, lógica de aplicação e manipulação de dados.

Padrão REST -

A comunicação entre a interface web e o backend será realizada utilizando API REST, permitindo que os recursos do sistema (clientes, produtos, vendas etc.) sejam acessados por meio de endpoints HTTP. As respostas da API serão retornadas no formato JSON, facilitando a integração entre os componentes do sistema.

Padrão de Autenticação com Token (JWT) -

Para garantir segurança no acesso ao sistema, será utilizado o padrão de autenticação baseado em JSON Web Token (JWT). Após o login do usuário, um token será gerado e utilizado para autenticar as requisições subsequentes à API, garantindo controle de acesso às funcionalidades do sistema.

Referências e Links



- Link do Projeto: <https://github.com/JiscoTorres/TheWay.git>
- GitHub - Marcus Vinícius R. Bacelar: <https://github.com/MarcusBacelar>
- GitHub - João Francisco Torres: <https://github.com/JiscoTorres>

FIELDING, Roy T.

Architectural Styles and the Design of Network-based Software Architectures.

Disponível em:

https://www.ics.uci.edu/~fielding/pubs/dissertation/rest_arch_style.htm

DJANGO SOFTWARE FOUNDATION.

Django Documentation.

Disponível em:

<https://docs.djangoproject.com/>

JWT.IO.

Introduction to JSON Web Tokens.

Disponível em:

<https://jwt.io/introduction>

HEUSER, Carlos Alberto. Projeto de Banco de Dados. 6. ed. Porto Alegre: Bookman, 2009.

DATE, C. J. Introdução a Sistemas de Bancos de Dados. Rio de Janeiro: Elsevier, 2004.

ELMASRI, Ramez; NAVATHE, Shamkant. Sistemas de Banco de Dados. 6. ed. São Paulo: Pearson, 2011.

